

Shortest Paths and Depth-First Search

With the machinery of graphs in hand — nodes, edges, digraphs, weights — we turn to *the* classic graph optimization problem: finding the cheapest way to get from one node to another, and the first algorithm that solves it.

Defining the Shortest Path

The **shortest path problem** asks for a path between two vertices such that the cost of traversing it is minimized. In the **unweighted** version, a shortest path from node n_1 to node n_2 is a sequence of edges satisfying three conditions:

1. The source node of the first edge is n_1 — you start where you say you start.
2. The destination of the last edge is n_2 — you finish where you want to go.
3. The sequence hangs together: if edge e_2 follows edge e_1 , then the source of e_2 is the destination of e_1 .
No teleporting — each edge picks up exactly where the previous one left off.

Here we simply count edges. Formally, a path in a graph is a sequence of vertices $P = (v_1, v_2, \dots, v_n)$ where each v_i is adjacent to v_{i+1} ; such a path has **length** $n - 1$ (the number of edges), and the unweighted problem is to minimize that length.

In the **weighted** version, each edge carries a real-valued weight via a function $f : E \rightarrow \mathbb{R}$ encoding distance, cost, or time, and we seek the path minimizing the total:

$$\sum_{i=1}^{n-1} f(e_{i,i+1}).$$

The two formulations connect neatly: when every edge has unit weight ($f : E \rightarrow \{1\}$), minimizing the weighted sum is exactly equivalent to finding the path with the fewest edges. The problem makes sense on undirected graphs (every edge traversable in either direction), directed graphs (consecutive vertices must be connected by an appropriately directed edge), and mixed graphs.

In the weighted digraph above, the shortest path from A to F is (A, C, E, D, F) — not necessarily the path with the fewest edges, because low-weight detours can beat direct-but-expensive hops. This is precisely why the weighted objective minimizes the *sum* rather than the count.

Variants of the Problem

The two-endpoint version just described is called the **single-pair shortest path problem**, to distinguish it from important generalizations:

- **Single-source**: shortest paths from one vertex to all others.
- **All-pairs**: shortest paths between every pair of vertices v, v' . For unweighted directed graphs this was introduced by Shimbel (1953), who showed it solvable by a linear number of matrix multiplications taking total time $O(V^4)$.

These generalizations admit significantly more efficient algorithms than naively running a single-pair solver on every relevant pair. There are also structural shortcuts: on arbitrarily-weighted **directed acyclic graphs**, a single-source solution comes in $\Theta(E + V)$ time using topological sorting — acyclicity removes the complications cycles introduce.

Where Shortest Paths Show Up

The problem is ubiquitous:

- **Routing and maps**: finding a route from one city to another — your GPS. A road network models naturally as a graph with positive weights: nodes are road junctions, edges are road segments, and

edge weights correspond to segment lengths. This is exactly what powers driving directions on web mapping sites like MapQuest or Google Maps, where fast specialized algorithms exist.

- **Communication networks:** getting a message from here to there with the fewest hops. In networking this is sometimes called the **min-delay path problem**, often studied alongside the widest path problem (e.g., seeking the shortest widest path or widest shortest path).
- **Physical and chemical spaces:** finding a path for a molecule threading through a lattice-like chemical labyrinth.
- **State-space search:** represent a nondeterministic abstract machine as a graph whose vertices are states and whose edges are transitions; shortest path algorithms find an optimal sequence of choices to reach a goal state, or establish lower bounds on the time needed. If vertices are configurations of a puzzle like a Rubik's Cube and edges are single moves, the shortest path is the minimum-move solution.
- **Social graphs:** lighthearted “six degrees of separation” games find shortest paths through graphs of movie stars appearing in the same film.
- **Operations research:** plant and facility layout, robotics, transportation, and VLSI design.
- **Network flows:** flow problems model moving goods, liquids, or information through a directed network where each edge has a capacity. For single-source, single-sink networks, the flow problem can be transformed into a series of shortest path problems — a useful reduction connecting the two areas.

A Worked Example: The Seven-City Digraph

To make things concrete, consider a directed graph over seven cities with ten edges:

From	To
Boston	Providence, New York
Providence	Boston, New York
New York	Chicago
Chicago	Denver, Phoenix
Denver	Phoenix, New York
Los Angeles	Boston
Phoenix	<i>(nothing)</i>

Two structural observations matter. First, the graph is genuinely **directed**: Boston–Providence exists in both directions, but New York → Chicago exists while Chicago → New York does not. Second, Phoenix has **no outgoing edges at all** — it is a sink; if you end up there, you're stuck.

A convenient textual encoding is the **adjacency list**: for each node, list its outgoing neighbors (Boston: Providence, New York; Providence: Boston, New York; New York: Chicago; Chicago: Denver, Phoenix; Denver: Phoenix, New York; Los Angeles: Boston; Phoenix: nothing).

The builder function `buildCityGraph` takes a `graphType` argument, so the same construction yields either a digraph or an undirected graph as desired. It proceeds in two phases: create the seven nodes by looping over the names Boston, Providence, New York, Chicago, Denver, Phoenix, Los Angeles and calling `g.addNode(Node(name))` for each; then issue ten `addEdge` calls in exactly the pattern above — crucially adding *both* Boston→Providence and Providence→Boston, but only New York→Chicago and never the reverse.

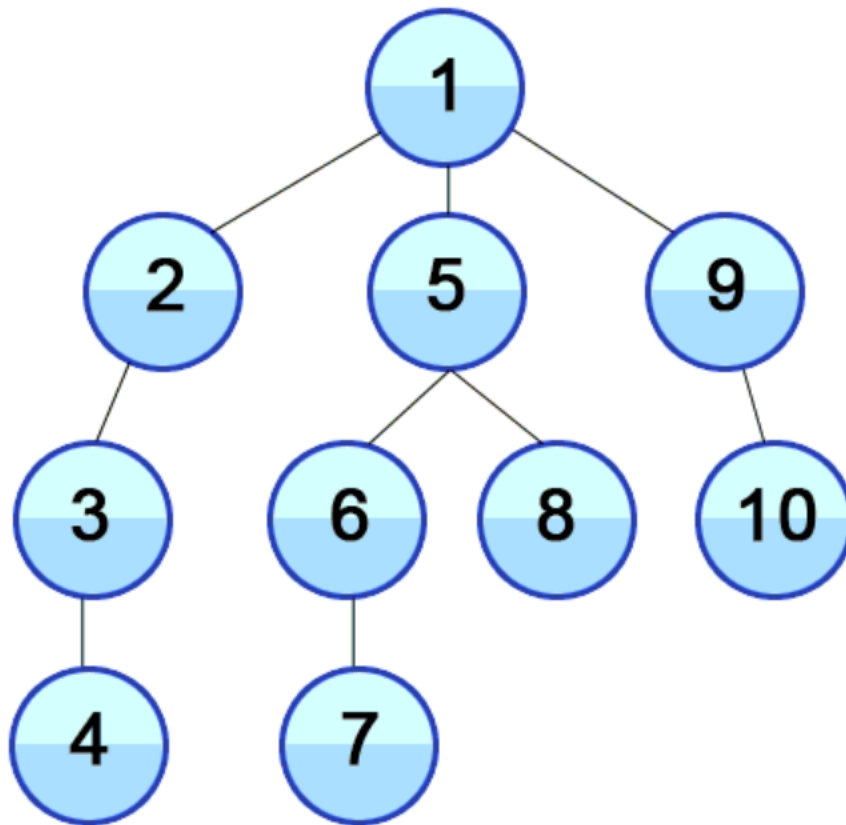
Depth-First Search: The First Path-Finding Algorithm

How do we actually *find* a shortest path in such a graph? The first algorithm is **depth-first search (DFS)**, analogous to the left-first depth-first method of enumerating a search tree — but with one critical difference: **a graph may have cycles**. Our city graph does: Denver → New York → Chicago → Denver. In a tree you can never return to a node; in a graph you can, so DFS **must keep track of already-visited nodes** to avoid walking in circles forever.

This failure mode is vivid: running DFS on a cyclic graph *without* remembering visited nodes produces an endless loop — in one standard example, the search visits A, B, D, F, E, then A again, cycling through A, B, D, F, E forever and never reaching parts of the graph it could otherwise reach. With memory of visited nodes, the same search cleanly visits every reachable node once, and the edges it traverses form a spanning structure known as a **Trémaux tree**. Iterative deepening — rerunning the search with increasing depth limits — is an alternative technique that sidesteps the infinite loop.

Mechanically, DFS starts at a root node (in a graph, some arbitrary starting node) and **explores as far as possible along each branch before backtracking**, using extra memory — typically a **stack** — to record the nodes discovered along the current branch so it can backtrack correctly. The idea is old: a version was investigated in the 19th century by Charles Pierre Trémaux as a strategy for solving mazes.

Its cost profile: traversing an entire graph takes $O(|V| + |E|)$ time — linear in the size of the graph — and $O(|V|)$ worst-case space for the stack plus the visited set. These bounds match breadth-first search, so the choice between them depends less on complexity than on the different vertex-ordering properties they produce. For very large or effectively infinite graphs (AI search, web crawling), a **limited-depth** variant searches only to a fixed depth without storing all visited vertices: time stays linear in the number of expanded vertices and edges, while space drops to be proportional only to the depth limit — far smaller than breadth-first search at the same depth — and the method lends itself well to heuristics for choosing promising branches. When no good depth limit is known in advance, **iterative deepening DFS** applies the limited search with a sequence of increasing limits; with branching factor greater than one, geometric growth of nodes per level means this costs only a constant factor over knowing the right limit upfront. One caveat: incomplete DFS, like incomplete BFS, is biased toward high-degree nodes if used to sample. Implementations come in recursive and explicit-stack forms, which visit each vertex's neighbors in opposite orders.



Source: Wikipedia, article “[Depth-first search](#)”.

The animation shows the characteristic behavior: plunge down one branch as deeply as possible, backtrack when stuck, and repeat — with visited-node bookkeeping preventing any revisit-driven looping.

The Divide-and-Conquer Insight

Hiding inside shortest-path search is a lovely recursive structure: **if we can find a path from the source to some intermediate node, and a path from that intermediate node to the destination, then their combination is a path from source to destination.** This composition property lets us break the big problem into smaller subproblems — and it is exactly the insight the DFS implementation exploits, which is where we pick up next.

From Hop Count to Total Cost: The Weighted Shortest Path Problem

Every shortest-path question so far measured distance in **edges** — the number of hops between two nodes. The **weighted** version changes the objective: each edge carries a cost, and we want the path that minimizes the *sum* of the weights of its edges, not their count. Formally, for an undirected simple graph G with a real-valued weight function $f : E \rightarrow \mathbb{R}$, the shortest path from v to v' is the path $P = (v_1, v_2, \dots, v_n)$ (with $v_1 = v$, $v_n = v'$, and each v_i adjacent to v_{i+1}) that minimizes

$$\sum_{i=1}^{n-1} f(e_{i,i+1}).$$

The canonical mental model is a road map: vertices correspond to intersections, edges to road segments, and each edge is weighted by the length or distance of its segment. This is why the professor’s driving intuition holds — you would happily take a route with *more turns* if its total distance is shorter. The cheapest path in total weight need not be the one with the fewest edges.

One structural fact ties the weighted and unweighted worlds together: when every edge has unit weight, i.e. $f : E \rightarrow \{1\}$, minimizing the weight sum is *equivalent* to finding the path with fewest edges. So the hop-count problem is just a special case, and the two objectives genuinely diverge only once weights vary. The problem is defined for undirected, directed, and mixed graphs alike, and it comes in variants: the basic **single-pair** form above, plus single-source and **all-pairs** versions. The all-pairs problem for unweighted directed graphs was introduced by Shimbel (1953), who showed it could be solved by a linear number of matrix multiplications taking $O(V^4)$ total time — and these generalized formulations admit significantly more efficient algorithms than naively running a single-pair solver on every relevant pair.

Why BFS Cannot Be Saved — and Why DFS Can Be Bent to the Task

BFS is mechanistically committed to hop count. It starts at the root and explores *all* nodes at the present depth before moving to the next depth level, holding the frontier in a queue; the result is a breadth-first tree whose parent links trace the shortest path — shortest in edges — back to the root, all in $O(|V| + |E|)$ time. Every piece of that machinery rests on the assumption that fewer edges is better. Introduce varying weights and the assumption breaks: a two-hop path can easily cost more than a three-hop path, and BFS, exploring strictly level by level, would hand you the expensive two-hop route without ever noticing the cheaper detour. Its ordering simply cannot see total accumulated weight.

DFS, by contrast, adapts readily: as you walk the graph, accumulate the weights along the way and keep track of the best total seen so far. That is a straightforward modification of a search you already have. For production use there are also dedicated algorithms — several well-known ones exist for the shortest path problem and its variants. A topological-sorting approach solves the single-source problem in $\Theta(E + V)$ on arbitrarily-weighted directed acyclic graphs, standard algorithm tables extend coverage to negative weights (where an algorithm may need to find a negative cycle or compute distances to all vertices), and road-network applications get fast specialized algorithms of their own.

The road-map setting makes the BFS limitation vivid. Take a map of cities connected by roads — exactly the graph model above — and run BFS from one city, say Frankfurt:

What comes out is a tree that layers every other city purely by its hop distance from the start:

That tree records *how many edges away* each city is, not *how far away* it is. Under weights, that is precisely the wrong bookkeeping — which is why BFS solves the unweighted problem, and weighted graphs need something else.

Where Shortest Paths Do Real Work

The weighted formulation powers a striking range of applications:

- **Driving directions.** Web mapping services like MapQuest and Google Maps run shortest path algorithms over road networks modeled as graphs with positive weights — nodes are road junctions, each edge is a road segment between junctions, and its weight is the segment’s length. Fast specialized algorithms exist for exactly this use case.
- **Abstract machines and puzzles.** Represent a nondeterministic abstract machine as a graph whose vertices are states and whose edges are possible transitions; shortest path algorithms then find an optimal sequence of choices to reach a goal state, or establish lower bounds on the time needed to get

there. The textbook example: vertices are configurations of a Rubik's Cube, each directed edge is a single move or turn, and a shortest path is a solution using the minimum possible number of moves.

- **Networking and telecommunications.** Here the problem appears as the *min-delay path* problem, usually studied alongside the *widest path* problem — one may seek the shortest (min-delay) widest path, or the widest shortest (min-delay) path.
- **Social graphs.** The “six degrees of separation” games are literally shortest path searches over graphs of movie stars appearing in the same film.
- **Operations research.** Plant and facility layout, robotics, transportation, and VLSI design all lean on shortest path computations.
- **Network flows.** Flow problems model transporting goods, liquids, or information through a network: a directed graph where each edge is a pipe, wire, or road with a *capacity* (the maximum amount that can flow through it), and the goal is a feasible flow maximizing source-to-sink throughput. Certain flow problems — particularly single-source, single-sink networks — can be transformed into a series of shortest path problems, so the machinery above feeds directly into flow computation.

Recap: Model It as a Graph, Then Search It

The lecture closes on two takeaways. First, **graphs are the right model for many things** — seriously. They capture *relationships among objects*, and a huge fraction of interesting problems is fundamentally about relationships. Better still, many important problems can be posed as graph optimization problems that are already solved: recognize that your problem has graph structure, and you inherit an entire algorithmic toolbox for free. Second, **depth-first and breadth-first search are the workhorses** — they find paths, check connectivity, and solve shortest paths in the unweighted sense, and as the weighted discussion shows, they are also the starting point for reasoning about the harder weighted versions.

The pedigree of these tools runs deep. BFS was invented in 1945 by Konrad Zuse, in his (rejected) Ph.D. thesis on the Plankalkül programming language — unpublished until 1972 — and reinvented in 1959 by Edward F. Moore specifically *to find the shortest path out of a maze*; C. Y. Lee later developed it into a wire routing algorithm (1961). Its robustness matters in practice: on implicitly represented graphs that are too large to store, or even infinite — game trees being the classic case — BFS is *complete*, guaranteed to find a goal state if one exists, whereas plain DFS may wander down an infinite branch and never return. (Iterative deepening DFS repairs that flaw at the price of re-exploring the tree's upper levels repeatedly, though both depth-first variants typically need far less extra memory than BFS's queue.) A chess engine illustrates the pattern: build the game tree from the current position by applying all possible moves, and BFS will find a winning position for White if one exists. In cost terms, BFS runs in $O(|V| + |E|)$ time and $O(|V|)$ space on explicit graphs; on implicit graphs, reaching nodes at distance d takes $O(b^{d+1})$ time and memory, where b is the branching factor.

So the story of the lecture in one line: model your problem as a graph, and then search it.